CSS3122 DATA STRUCTURE AND ALGORITHMS

CH3 USING FUNCTION WITH STRUCTURE

KHAIRUNNISA IBR

CHAPTER OUTLINE

In this chapter, you will learn about:

1. What is function?
2. Types of function
3. How function works
4. What is structure?
5. What is typedef?
6. Structures & functions

WHAT IS FUNCTION?

DEFINITION

- Functions allow structure program in segments of code to perform individual tasks.
- It is exactly the same as mathematical functions.
- Syntax of function:

type name (parameter1, parameter2, …) { statements }

- Type - type of the value returned by the function.
- Name - identifier by which the function can be called.
- Each parameter - consists of a type followed by an identifier, with each parameter separated by a comma.
- Statements - function's body surrounded by braces.

WHAT IS FUNCTION? (CONT…)

A RECAP…

```
#include <stdio.h>
int myfunc(int a); //what is this called?
Int b; //global variable
int main() {
    int result = myfunc(10); //what is this called?
    printf("The result is %d\n",result);
B=result;
    return 0;
}
int myfunc(int a) {//what is this called?
B=a;
    return -a;
}
```

What is **local** variable?

What is **global** variable?

TYPES OF FUNCTION

There are 2 categories of functions:

Predefined functions	User-defined functions
• Obtained from libraries.	• Written by programmers themselves.

TYPES OF FUNCTION (CONT…)

Predefined functions

- Already defined in libraries.
- Existing code that can be re-used in the system libraries.
- Helpful to write error-free code.
- But user must know the syntax of the function.

e.g. 1 **printf() and scanf() is available in stdio.h library.**

e.g. 2 **sqrt() function is provided in math.h library.**
Its usage is y=sqrt (x), where x must be positive.
If x is 25, i.e. y = sqrt (25) then y = 5.

e.g. 3 **clrscr() is available in conio.h library.**

TYPES OF FUNCTION (CONT…)

```
#include<stdio.h>
#include<math.h>
int main ()
{
   int x,y;
   printf (“enter a positive number”);
   scanf (“ %d”, &x);
   y = sqrt(x);
   printf(“squareroot = %d”, y);
   return 0;
}
```

OUTPUT

Enter a positive number 25

Squareroot = 5

TYPES OF FUNCTION (CONT…)

User-defined functions

- must be defined by the programmer or user.
- Programmer has to write the coding for such functions and test them properly before using them.
- The syntax of the function is given by the user so there is no need to include any header files.
- **e.g. 1 main(), swap(), sum(), etc.,**

TYPES OF FUNCTION (CONT…)

```
#include<stdio.h>
int sum (int, int);
int main (){
   int a, b, c;
   printf (“enter 2 numbers”);
   scanf (“ %d %d”, &a ,&b);
   c = sum (a,b);
   printf(“sum = %d”, c);
   return 0;
}
```

```
int sum (int a, int b)
{
   int c;
   c=a+b;
   return c;
}
```

OUTPUT

Enter 2 numbers 10 20

Sum = 30

HOW FUNCTION WORKS

- The execution of a C program begins from the **main()** function.

```
#include <stdio.h>
void functionName()
{
   …
}

int main()
{
   …
   functionName();
   …
}
```

- When the compiler encounters **functionName();** control of the program jumps to **void functionName()**
- And, the compiler starts executing the codes inside **functionName().**
- The control of the program jumps back to the **main()** function once code inside the function definition is executed.

HOW FUNCTION WORKS

- How function works in C programming?

```
#include <stdio.h>

void functionName()
{
    ... .. ...
    ... .. ...
}

int main()
{
    ... .. ...
    ... .. ...

    functionName();

    ... .. ...
    ... .. ...
}
```

WHAT IS STRUCTURE?

- A collection of variables under a single name.
- The variables in a structure are of different data types and each has a name that is used to select it from the structure.

Syntax

```
struct structName
{
    datatype varName;
    datatype varName;
    …
    …
};
int main()
{
    …
    struct structName structVar;
```

For example:

```
struct Student
{
    int studentID;
    char name[20];
    char course[7];
    float fees; //what are these called?
};
int main()
{
    …
    struct Student stud1; //what is this called?
```

WHAT IS STRUCTURE? (CONT…)

A RECAP…

- There are 2 possible ways to declare variables of a structure:

Method 1

```
struct Student
{
    int studentID;
    char name[20];
    char course[7];
    float fees;
};
int main()
{
    …
    struct Student stud1;
```

Can also be written as…

Method 2

```
struct Student
{
    int studentID;
    char name[20];
    char course[7];
    float fees;
} stud1;
int main()
{
    stud1= {…
    …
```

What if there are more than 1 structure variables to declare?

If many variables?

```
struct Student
{
    int studentID;
    char name[20];
    char course[7];
    float fees;
} stud1, stud2;
int main()
{
    stud1= {…
    stud2= {…
    …
```

WHAT IS TYPEDEF?

- Means "type definition".
- Enable programmer to create a new data type name using an existing data type.
- No new data is created, but alternate name is given to a known data type.
- Syntax:

typedef existing_data_type new_data_type_name;

- E.g. **typdef int INTEGER;** //definition of new type

  INTEGER num=5; //declaring variable of the new type

WHAT IS TYPEDEF?

- Using typedef with struct turns the struct into a new type.
- It is used to make the construct shorter with more meaningful names.
- Example:

Normal struct construct

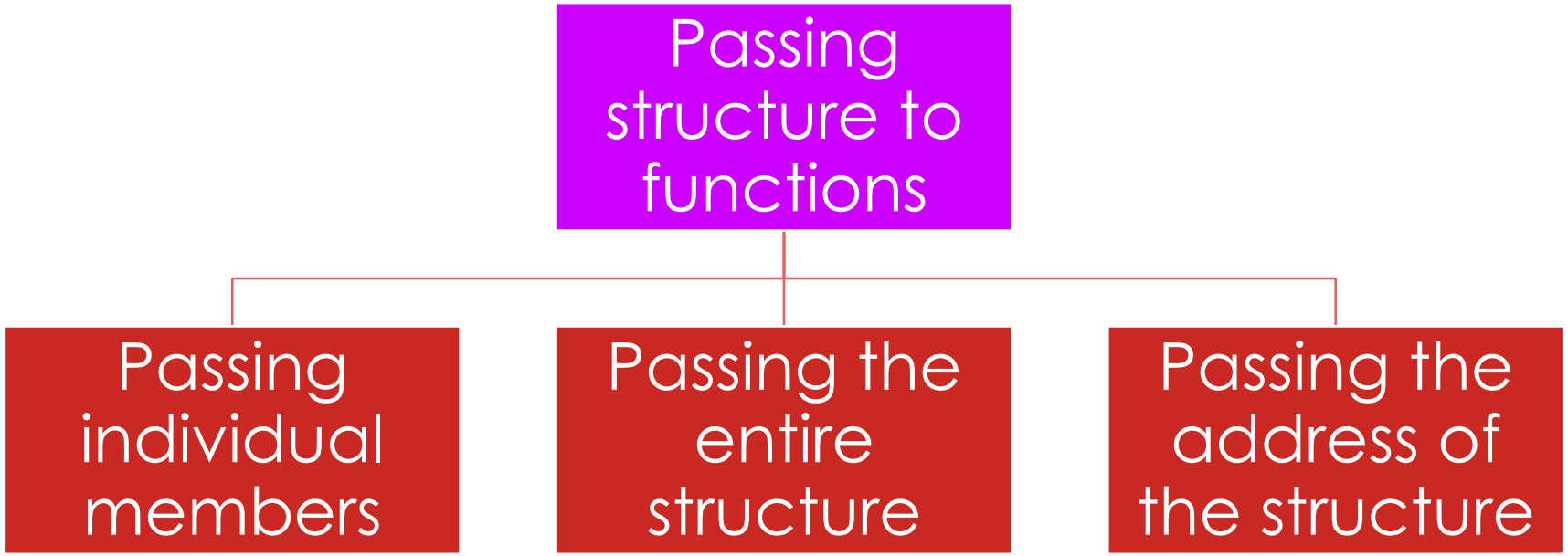
```
struct Student
{
    int studentID;
    char name[20];
    char course[7];
    float fees;
};
int main()
{
    struct Student stud1;
```

Using typedef with struct

```
typedef struct Student //typedef is used
{
    int studentID;
    char name[20];
    char course[7];
    float fees;
};
int main()
{
    Student stud1; //shorter
```

STRUCTURES AND FUNCTIONS

- We must have a mechanism to pass structures to functions and return them.
- A function may access structure members in three ways:



STRUCTURES AND FUNCTIONS (CONT…)

PASSING INDIVIDUAL MEMBERS

- To pass individual structure members to a function, we must use direct selection operator **(structVar.structMember)** to refer to the members.

```
#include<stdio.h>
typedef struct
{
   int x;
   int y;
}POINT;
void display(int, int);
```

```
int main()
{
   POINT p1 = {2,3};
   display(p1.x, p1.y);
   return 0;
}
```

```
void display(int a, int b)
{
   printf("The coordinates of the point are: %d %d", a, b);
}
```

OUTPUT

The coordinates of the point are: 2 3

STRUCTURES AND FUNCTIONS (CONT…)

PASSING THE ENTIRE STRUCTURE

- When a structure is passed as an argument, its is passed using the call by value method,
- i.e. a copy of each of the structure is made.
- Syntax of passing a structure to a function and returning a structure:

```
struct structName functName(struct structName structVar);
```

- The syntax above may vary if the return data type is different or void.

STRUCTURES AND FUNCTIONS (CONT…)

PASSING THE ENTIRE STRUCTURE (cont…)

- Example of a function that receive a structure and return void:

```
#include<stdio.h>
typedef struct
{
   int x;
   int y;
}POINT;
void display(POINT);
```

```
int main()
{
   POINT p1 = {2,3};
   display(p1);
   return 0;
}
```

```
void display(POINT p)
{
   printf("The coordinates of the point are: %d %d", p.x, p.y);
}
```

OUTPUT

The coordinates of the point are: 2 3

STRUCTURES AND FUNCTIONS (CONT…)

PASSING STRUCTURE THROUGH POINTERS

- Passing large structures to functions using the call by value method is very inefficient.
- It is preferred to pass structures through pointers.
- A pointer to a structure holds the address of the structure.
- Syntax to declare a pointer to a structure:

```
struct structName
{
   dataType member1;
   dataType member2;
   …
};
```

```
int main()
{
   struct structName
*ptr;
```

OR

```
struct structName
{
   dataType member1;
   dataType member2;
   …
}*ptr;
```

STRUCTURES AND FUNCTIONS (CONT…)

PASSING STRUCTURE THROUGH POINTERS (cont…)

- After declaration of the pointer variable, e.g.

```
struct student *ptr_stud;
```

- we need to assign address of stud to the pointer using the address operator (&), e.g

```
ptr_stud=&stud;
```

- To access the members of a structure, we can write:

```
ptr_stud -> roll_no = 123;
//assign values to the roll_no in the structure ptr_stud points to
//OR
(*ptr_stud).roll_no=123;
```